# 如何生成与解析稳定的 JSON 格式

在大模型（LLM）应用开发中，JSON 格式是数据交互、函数调用（Function Calling）以及结构化信息提取的核心载体。然而，大模型本质上是基于概率的文本生成器，尽管目前业界顶尖模型在遵循格式方面进步显著，但在复杂场景或部分开源/国内模型中，依然会频繁出现 JSON 结构不规范、包含冗余文本、输出被截断等问题。

为了构建健壮的 LLM 应用，我们需要从**“事前预防（Prompt与模型层面）”和“事后兜底（后处理与修复解析）”**两个维度来全面保障 JSON 输出的稳定性。

以下是一份从测试、生成到修复的完整实战指南。

---

## 第一部分：事前预防（提升模型原生输出质量）

在依赖复杂的正则表达式或后处理脚本之前，最好的策略是尽可能让模型一次性输出正确的格式。

### 1. 强化 Prompt 设计 (Prompt Engineering)

- **明确的指令约束：** 在 System Prompt 或 User Prompt 的末尾，使用强烈且明确的指令。例如：“请务必仅输出纯粹的 JSON 格式文本，不要包含任何 Markdown 标记（如 ```json ），也不要包含任何介绍或总结性的自然语言。”
- **提供 Few-Shot 示例：** 给模型提供 1-2 个输入和完美 JSON 输出的示例。这能大幅降低模型产生幻觉或自创格式的概率。
- **给出严格的 JSON Schema：** 直接在 Prompt 中定义你期望的 JSON 结构，并解释每个字段的含义和数据类型。

### 2. 利用模型原生特性 (Native Capabilities)

- **开启 JSON Mode / 结构化输出：** 目前主流模型（如 Gemini, GPT-4 等）大多通过 API 提供了原生 `response_format: { "type": "json_object" }` 甚至更严格的 Structured Outputs 功能。这能在底层强迫模型按照给定的 Schema 生成内容，极大减少语法错误。
- **选择合适的 Temperature：** 对于需要严格格式的数据提取任务，调低 `Temperature`（例如 0 到 0.3）可以减少生成的随机性，从而提高 JSON 的稳定性。

---

## 第二部分：压力测试与问题定位

在实际生产环境中，即便做了完善的 Prompt 约束，依然会有漏网之鱼。我们需要通过系统性的测试把潜在的格式问题提前“逼”出来。

### 1. 构建测试数据集

准备一批覆盖实际业务场景的测试 Prompt 集合。

### 2. 放大模型随机性进行压测

- **调高 Temperature：** 将大模型的 `Temperature` 调到较高值（如 `1.0` 甚至更高），主动促使模型产生幻觉或格式错误。

- **多轮与并发测试：** 每个 Prompt 跑多组结果（例如跑5-10轮）。利用 Python 的 `concurrent.futures.ThreadPoolExecutor` 进行多线程并发调用，快速收集大量的输出结果。
- **持久化失败 Case：** 封装一个安全的 JSON 检查函数（如使用 `try-except json.loads(text)`），将所有抛出 `JSONDecodeError` 的原始文本和错误原因保存到本地文件（如 `failed_outputs.json`）中，作为后续编写修复策略的素材库。

---

## 第三部分：常见解析失败 Case 及修复策略 (事后兜底)

针对压测收集到的失败 Case，我们可以通过编写字符串处理和正则表达式函数来进行自动修复。以下是业界常见的错误类型及应对思路：

### 1. 包含无关的自然语言文本

模型经常在 JSON 前后加上“这是您要的JSON：”或“希望对您有帮助！”等废话，或者将其包裹在 Markdown 代码块中。

- **修复思路：**
  - **去除 Markdown：** 使用正则 `re.sub(r"```(json)?", "", text)` 清理代码块标记。
  - **提取核心区块：** 假设你的 JSON 顶层是一个对象 `{...}` 或数组 `[...]`，可以通过 `text.find('{')` 和 `text.rfind('}')` 找到第一个左括号和最后一个右括号，然后将中间的内容（包含括号本身）提取出来。

### 2. 引号使用错误

JSON 标准严格要求键（Key）和字符串值（Value）必须使用**双引号**。

- **混合使用引号 / 单引号：** 模型可能混用单双引号，或者全部使用了单引号。可以通过脚本将特定模式下的单引号替换为双引号（需注意不要破坏字符串内部合法的单引号）。
- **未加引号的值：** 某些本应是字符串的值（如包含空格或特殊字符）未加引号。可以尝试用正则匹配 `key: value` 结构，强行为缺失引号的 value 补充双引号。

### 3. 键值对格式不正确

- **缺少冒号 / 使用等号：** 有时模型会输出 `"name" = "Alice"` 或 `"name" "Alice"`。
- **修复思路：** 使用正则检测类似 `"key" "value"` 或 `"key"="value"` 的模式，强制将其替换为标准的 `"key": "value"`。

### 4. 逗号的遗漏与冗余

- **多余逗号：** 常见于数组或对象的最后一个元素后（如 `{"a": 1, "b": 2, }`），很多解析器会报错。可以使用正则匹配并移除 `}` 或 `]` 前的尾随逗号。
- **遗漏逗号：** 两个键值对之间缺少逗号。这种情况修复难度稍大，需利用正则定位相邻的键值对进行补充。

### 5. 括号不平衡与输出截断

当生成的 token 达到上限时，JSON 对象或数组常常未闭合（例如缺少末尾的 `}` 或 `]`）。

- **修复思路：** 统计左花括号/方括号与右花括号/方括号的数量差。
  - 如果左侧多，则在字符串末尾补齐相应数量的右括号（如 `text += '}' * diff`）。

    - 如果右侧多，则尝试从末尾移除多余的括号。

### 6. 不可见字符污染

某些模型会在输出中夹杂不可见的零宽字符（如 `\u200b`, `\u200c` 等），导致解析器报错。

- **修复思路：** 在尝试解析前，做一次简单的全局替换，移除这些常见的不可见字符。

---

## 第四部分：构建健壮的 JSON 修复流 (Pipeline)

在实际应用中，我们不应该盲目应用所有的修复规则，因为某些规则（如粗暴的正则替换）可能会误伤原本正常的文本内容。应该构建一个**渐进式、带短路的重试管线**：

1. **直接尝试解析：** 拿到文本第一步永远是直接 `json.loads()`。如果成功，代表无需修复，直接返回。
2. **清理外层噪音：** 如果失败，先执行最安全的修复——去除 Markdown 标记和前后冗余文本，再次尝试解析。
3. **修复符号错误：** 接着按顺序尝试修复不可见字符、修复多余的尾随逗号、修复缺失/多余的括号。
4. **处理截断问题：** 如果依然失败，尝试进行括号计数的最终补齐（处理因 token 上限被截断的情况）。
5. **高风险修复：** 最后才尝试进行高风险的正则替换（如修复键值对等号、单双引号替换等）。每执行一步高风险操作后，都尝试一次 `json.loads()`，成功即跳出。
6. **最终兜底：** 如果所有修复策略都耗尽依然无法解析，抛出异常，触发重试机制（重新调用大模型）或回退至默认值。

### 总结

生成稳定的 JSON 不仅仅是写几句 Prompt 的事情，它是一项系统工程。**Prompt 与模型特性的利用是“防范于未然”，而完善的后处理管线则是“兜底保平安”**。将这两者结合，你就能在复杂的业务场景中获得高度可靠的结构化数据输出。

*提示：在 Python 生态中，目前也有诸如 `json_repair` 或 LangChain 的 `OutputParser` 等成熟的开源库，它们内置了上述提及的多种修复逻辑，在开发时可以直接引入使用，避免重复造轮子。*